



## Nextflow workflows fundamentals

# Introduction to Nextflow and Workflow Concepts

Understand the main parts: processes, channels, and workflow blocks in Nextflow scripts.

Learn to define simple process blocks for task execution in Nextflow workflows.

Follow best practices for structuring scripts to improve readability and maintenance.

# Workflows

**Channels** are the "pipes" that connect processes, carrying data from one step to the next.

**Processes** are atomic units that can be combined in different workflows, promoting reusability.

**Workflow as Orchestrator:** Workflows don't do computation themselves - they orchestrate processes and manage data flow through channels.

# Workflows

- Channels are the pipes between processes: here `samples_ch` is the input channel and `generated.out` is the output channel used by the downstream process.
- A process input block describes the shape expected (e.g., `val`, `path`, `tuple`) and the output block describes what it emits.

```
process generate_samples {
  input:
  val sample_id
  output:
  tuple val(sample_id), path("${sample_id}.txt"), emit: sample_file
  script:
  """
  printf "This is ${sample_id}\\nline2 of ${sample_id}\\nline3 of ${sample_id}\\n" > ${sample_id}.txt
  """
}

process count_lines {
  input:
  tuple val(sample_id), path(sample_file)
  output:
  stdout
  /*
  The script counts lines in the input file and prints a simple report.
  Each tuple results in one independent invocation of this process.
  */
  script:
  """
  echo "${sample_id}: $(wc -l < ${sample_file}) lines"
  """
}

workflow {
  // Create a channel of sample IDs (this is the entry point of the workflow)
  samples_ch = Channel.from('sampleA', 'sampleB', 'sampleC')

  // Pass the channel into the first process. Nextflow will call `generate_samples`
  // once for each item in samples_ch.
  generated = generate_samples(samples_ch)

  // Connect the output of generate_samples to count_lines. `generated.out` is the channel
  // that emits tuples (sample_id, path_to_file) produced by generate_samples.
  counted = count_lines(generated)

  // Print the results to the console. view() subscribes to the output channel.
  counted.view()
}
```

# Workflows

- Channels are the pipes between processes: here `samples_ch` is the input channel and `generated.out` is the output channel used by the downstream process.
- A process input block describes the shape expected (e.g., `val`, `path`, `tuple`) and the output block describes what it emits.
- Nextflow schedules process tasks independently for each element in a channel demonstrating data-parallel execution. The workflow block explicitly wires channels and processes together; this makes dependencies and data flow clear.

```
bruno.ariano@HM-H1XWYG3KQF ~/work/HT
• (base) > $ nextflow run test.nf
Nextflow 25.10.3 is available – Please consider updating your version to it

NEXTFLOW ~ version 25.10.2

Launching `test.nf` [cheeky_mendel] DSL2 – revision: 5e1192015e

executor > local (6)
[7f/40a5c9] generate_samples (1) [100%] 3 of 3 ✓
[c2/e477dd] count_lines (3) [100%] 3 of 3 ✓
sampleC:      3 lines

sampleB:      3 lines

sampleA:      3 lines


bruno.ariano@HM-H1XWYG3KQF ~/work/HT
• (base) > $ nextflow run test.nf
Nextflow 25.10.3 is available – Please consider updating your version to it

NEXTFLOW ~ version 25.10.2

Launching `test.nf` [spontaneous_hodgkin] DSL2 – revision: 5e1192015e

executor > local (6)
[1c/c758e3] generate_samples (3) [100%] 3 of 3 ✓
[c2/9f5206] count_lines (3) [100%] 3 of 3 ✓
sampleB:      3 lines

sampleA:      3 lines

sampleC:      3 lines
```

# Workflows

- Depending on the type of channel some are recycled while other are consumed.
- If 2 channel of different dimension are given to a process the one shorter dictate when the process terminates.

```
process echo {  
  input:  
  val x  
  val y  
  
  output:  
  stdout  
  
  script:  
  ""  
  
  echo $x and $y  
  ""  
}
```

```
workflow {  
  x = channel.value(1)  
  y = channel.of('a', 'b', 'c')  
  echo(x, y).view()  
}
```

```
executor > local (3)  
[9d/b1892f] echo (1) [100%] 3 of 3 ✓  
1 and c  
  
1 and b  
  
1 and a
```

```
process echo {  
  input:  
  path x  
  val y  
  
  output:  
  stdout  
  
  script:  
  ""  
  
  echo $x and $y  
  ""  
}
```

```
workflow {  
  x = channel.fromPath(["traits_filter_0T_1K.csv"])  
  y = channel.of('a', 'b', 'c')  
  echo(x, y).view()  
}
```

```
executor > local (1)  
[5a/55ca92] echo (1) [100%] 1 of 1 ✓  
traits_filter_0T_1K.csv and a
```

# Workflows

- Depending on the type of channel some are recycled while other are consumed.
- If 2 channel of different dimension are given to a process the one shorter dictate when the process terminates.
- To overcome this issue, we can combine both channel

```
process echo {  
  input:  
    tuple path(x), val(y)  
  
  output:  
    stdout  
  
  script:  
    """  
    echo $x and $y  
    """  
}  
  
workflow {  
  x = channel.fromPath(["traits_filter_0T_1K.csv"])  
  y = channel.of('a', 'b', 'c')  
  combine = x.combine(y)  
  echo(combine).view()  
}
```

```
executor > local (3)  
[a9/ce4043] echo (2) [100%] 3 of 3 ✓  
traits_filter_0T_1K.csv and c  
  
traits_filter_0T_1K.csv and a  
traits_filter_0T_1K.csv and b
```

# Workflows

- Depending on the type of channel some are recycled while other are consumed.
- If 2 channel of different dimension are given to a process the one shorter dictate when the process terminates.
- To overcome this issue, we can combine both channel

```
process echo {  
  input:  
    tuple path(x), val(y)  
  
  output:  
    stdout  
  
  script:  
    """"  
    echo $x and $y  
    """"  
}  
  
workflow {  
  x = channel.fromPath(["traits_filter_0T_1K.csv"])  
  y = channel.of('a', 'b', 'c')  
  combine = x.combine(y)  
  echo(combine).view()  
}
```

```
executor > local (3)  
[a9/ce4043] echo (2) [100%] 3 of 3 ✓  
traits_filter_0T_1K.csv and c  
  
traits_filter_0T_1K.csv and a  
traits_filter_0T_1K.csv and b
```



# Workflows

- Depending on the type of channel some are recycled while other are consumed.
- If 2 channel of different dimension are given to a process the one shorter dictate when the process terminates.
- To overcome this issue, we can combine both channel

```
process echo {  
  input:  
    tuple path(x), val(y)  
  
  output:  
    stdout  
  
  script:  
    ""  
    echo $x and $y  
    ""  
}  
  
workflow {  
  x = channel.fromPath(["traits_filter_0T_1K.csv"])  
  y = channel.empty()  
  combine = x.combine(y)  
  echo(combine).view()  
}
```

```
Launching `main.nf` [astounding_cuvier] DSL2 - revision: 329f4ae9b6  
[-- ] echo -
```

# Workflows

- Multiple output can be passed from a process to the next using emit

```
process PROCESS_DATA {
  input:
  val sample_id

  output:
  path "${sample_id}.txt", emit: data_file
  val sample_id, emit: sample_name

  script:
  """
  echo "Data for ${sample_id}" > ${sample_id}.txt
  """
}

process ANALYZE_RESULTS {
  input:
  path data_file
  val sample_name

  output:
  path "${sample_name}_report.txt"

  script:
  """
  echo "Analysis Report for ${sample_name}" > ${sample_name}_report.txt
  echo "Data: \$(cat ${data_file})" >> ${sample_name}_report.txt
  """
}

workflow {
  samples = Channel.of('sample1', 'sample2')

  PROCESS_DATA(samples)

  ANALYZE_RESULTS(
    PROCESS_DATA.out.data_file,
    PROCESS_DATA.out.sample_name
  )
}
```

# Hands-on

We are going to create a workflow that perform QC and alignment on sequencing reads using the programs fastp, bwa and samtools.



**OpenGene - Open Source Genomics Toolbox**

Open source tools for NGS data analysis

**fastp** is a **FASTQ pre-processing and quality control tool** used in sequencing. Its main functions are: Quality filtering & trimming, Adapter trimming, Paired-end handling, Base correction, Quality control reporting.

**bwa** is a software package for mapping low-divergent sequences against a large reference genome, such as the human genome.

**Samtools** is a suite of programs for interacting with high-throughput sequencing data. It consists of three separate repositories. We are only interested in tool of Samtools that merge paired-ends reads for each sample.

BWA (<https://bio-bwa.sourceforge.net/>)

SAMTOOLS (<https://www.htslib.org/>)

FASTP (<https://github.com/OpenGene/fastp>)

